

Code-Layout, Readability and Reusability

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Evaluation of the codebase's structure, readability, and reusability, updated to reflect the authentication, database, and server-side validation layers added to the application since the initial submission.

Code Layout Checklist

S.N O	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing used throughout the code	Yes	Standard 4-space Python indentation across app.py, models.py, forms.py and extensions.py
2	Proper File Structure	Files and folders are logically organized	Yes	app.py, extensions.py, models.py and forms.py at root; templates/ (with partials/ and errors/ subfolders) and static/ follow Flask convention
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	FEATURE_COLUMNS, GOOGLE_LOGIN_ENABLED, MODEL_READY, flood_predicted, etc. map directly to their role
4	Function / Method Names	Functions are descriptively named	Yes	home(), register(), login(), login_google(), google_callback(), logout(), predict(), history(), delete_prediction(), not_found(), server_error()
5	Code Comments	Inline and block comments explain logic where needed	Yes	Comments added for proxy/HTTPS handling, model-loading fallback, Google account linking and open-redirect protection on login
6	Modular Design	Code is split into reusable functions/modules	Yes	Flask extensions isolated in extensions.py, ORM models in models.py, form validation in forms.py; app.py routes grouped by concern (public, auth, prediction, history, errors)
7	No Redundant Code	Duplicate or unused code is removed	Yes	No duplicate templates; shared partials/nav.html, partials/flash.html and partials/footer.html are reused instead of being copy-pasted per page
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Custom 404/500 error pages, try/except around model loading and the Google OAuth callback, server-side form validation errors, and flash-message feedback on failed predictions

S.N O	Code Quality Parameter	Description	Followed	Remarks
9	Security Practices	Sensitive data and requests are protected	Yes	CSRF protection (Flask-WTF) on every form, hashed passwords (Werkzeug), secrets loaded from environment variables via python-dotenv, and an open-redirect guard on the login 'next' parameter

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	header_table() navbar + theme toggle markup (partials/nav.html)	Jinja2 / HTML/CSS/JS	Included identically in home, predict, chance, no_chance, login, register and history templates	High
2	initTheme() / initRain() / initGauge() / initLightning()	JavaScript (main.js)	Shared across every page via one main.js file	High
3	CSS design tokens (:root variables)	CSS3	Drives both dark and light themes from a single stylesheet	High
4	db, login_manager, csrf extension instances	Python (extensions.py)	Imported into app.py, models.py and forms.py to avoid circular imports and duplicated setup	High
5	partials/flash.html message renderer	Jinja2 / HTML	Included on every page template to surface flash() messages consistently	High
6	WTForms validators (DataRequired, NumberRange, Email, EqualTo)	Python (forms.py)	Reused across RegistrationForm, LoginForm and PredictForm	High
7	Prediction DataFrame assembly block	Python (app.py)	Single predict() route; would be extracted into a helper if more model versions are added	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean separation of routes, models, forms, extensions, templates and static assets.
Readability	4	Clear naming; comments now cover the trickier logic (proxy handling, OAuth linking, redirect safety).
Reusability	5	Shared CSS/JS/partials reused across all seven page templates; extensions module removes duplicated Flask setup code.
Documentation / Comments	3	README.md and AUTH_SETUP.md cover setup, routes and the auth flow; inline comments remain intentionally sparse.
Overall Score	4	Solid, maintainable structure that now also covers authentication, persistence and validation.

Supporting Screenshots

Space reserved below for screenshots illustrating the project folder structure and code organization described above.

[Insert screenshot: project folder / file structure]

[Insert screenshot: app.py route organization]